

Machine Learning Assignment 2 — Report

Course: Machine Learning

Assignment: A02 — Unsupervised & Probabilistic Learning + Decision Trees

Roll Number: 23122 (Seed = 122)

Table of Contents

1. [Experimental Setup](#)
 2. [Question 1: k-Means Clustering](#)
 3. [Question 2: Gaussian Naive Bayes](#)
 4. [Question 3: Decision Trees \(C4.5\)](#)
 5. [Dataset Analysis Summary](#)
 6. [Implementation Notes](#)
 7. [Conclusion](#)
-

1. Experimental Setup

1.1 Allowed & Prohibited Libraries

- **Allowed:** NumPy, Pandas, Matplotlib
- **Allowed (limited):** scikit-learn for `train_test_split` and evaluation metrics only
- **Prohibited:** scikit-learn classifiers, clustering APIs, decision tree APIs, Naive Bayes APIs

1.2 Reproducibility

All experiments use a fixed random seed:

```
SEED = 122 # last 3 digits of roll number
np.random.seed(SEED)
```

1.3 Dataset Constraints Satisfied

Constraint	Status
Min samples ($n \geq 1000$)	✓ All datasets
Min features ($d \geq 15$)	✓ All datasets
Informative features (≥ 5)	✓ All datasets
Noisy features (≥ 5)	✓ All datasets
At least one dataset with $d \geq 50$	✓ <code>high_dimensional</code> ($d=60$)
At least one dataset with $n \geq 5000$	✓ <code>low_noise</code> & <code>high_dimensional</code> ($n=5000$)
Three dataset categories	✓ Low-noise, High-noise, High-dimensional

1.4 Datasets Generated

Dataset	n	d	Informative	Noise	Purpose
<code>low_noise</code>	5000	20	15	5	General benchmarking (well-separated Gaussians)
<code>high_noise</code>	3000	25	10	15	Robustness testing (heavy noise, overlapping classes)
<code>high_dimensional</code>	5000	60	10	50	Curse of dimensionality & overfitting
<code>kmeans_friendly</code>	2000	15	10	5	Q1-B(a): spherical clusters
<code>kmeans_adversarial</code>	2000	15	5	10	Q1-B(b): elongated anisotropic clusters
<code>nb_correlated</code>	2000	15	5	10	Q2-B: correlated features

Dataset	n	d	Informative	Noise	Purpose
nb_success	2000	15	5	10	Q2-C(a): NB works despite violated independence
nb_failure	2000	15	5	10	Q2-C(b): 5D parity, NB fails
tree_friendly	3000	20	10	10	Q3-B/C/E: axis-aligned boundaries
greedy_counterexample	2000	15	5	10	Q3-D: 5D parity, greedy splitting fails
tree_noisy	3000	20	10	10	Q3-E: label noise + outliers
overfitting	3000	25	5	20	Q3-C: overfitting demonstration

2. Question 1: k-Means Clustering [30 Marks]

2.1 Part A — Core Implementation

Implemented a fully vectorized k-Means algorithm with the following components: -

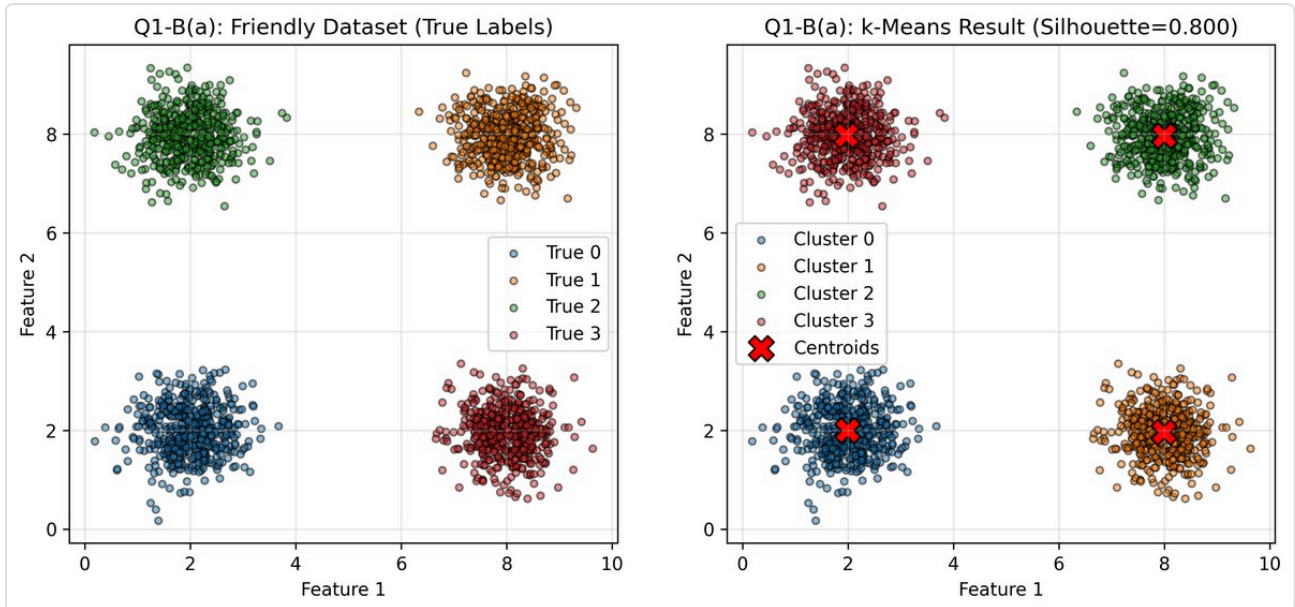
`random_centroid_init` : picks k unique data points as initial centroids - `compute_distances` : vectorized Euclidean distance using NumPy broadcasting $(n, 1, d) - (1, k, d)$ - `assign_clusters` : argmin across distance matrix - `update_centroids` : mean of assigned points, with empty-cluster re-initialization - `compute_cost` : sum of squared distances (inertia) - `fit` : Lloyd's algorithm with convergence tolerance $1e-4$ and max 300 iterations

Key design choice: All distance computations use NumPy vectorization; no Python loops for the core assignment step.

2.2 Part B — Adversarial Dataset Construction

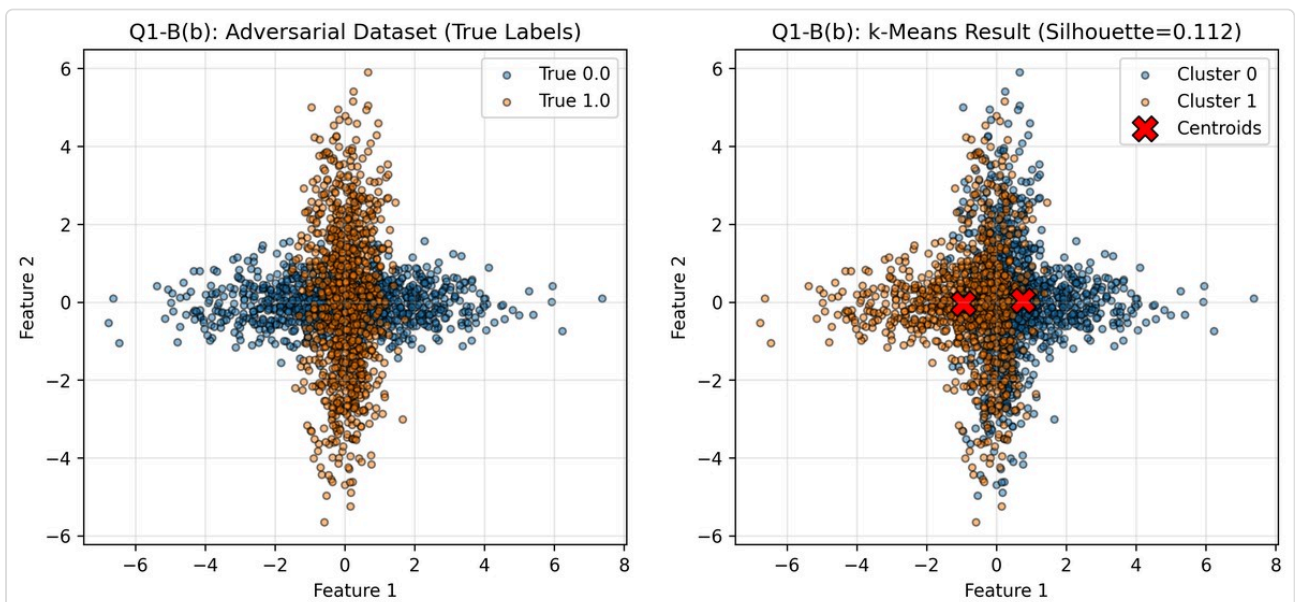
(a) Friendly Dataset — k-Means Performs Well

- **Structure:** 4 spherical Gaussian clusters with equal variance ($\sigma=0.5$), balanced sizes, clear separation
- **Result:** Silhouette Score = **0.8003**
- **Analysis:** All k-Means assumptions satisfied: spherical clusters, equal variance, balanced sizes. Centroids align perfectly with true cluster centers.



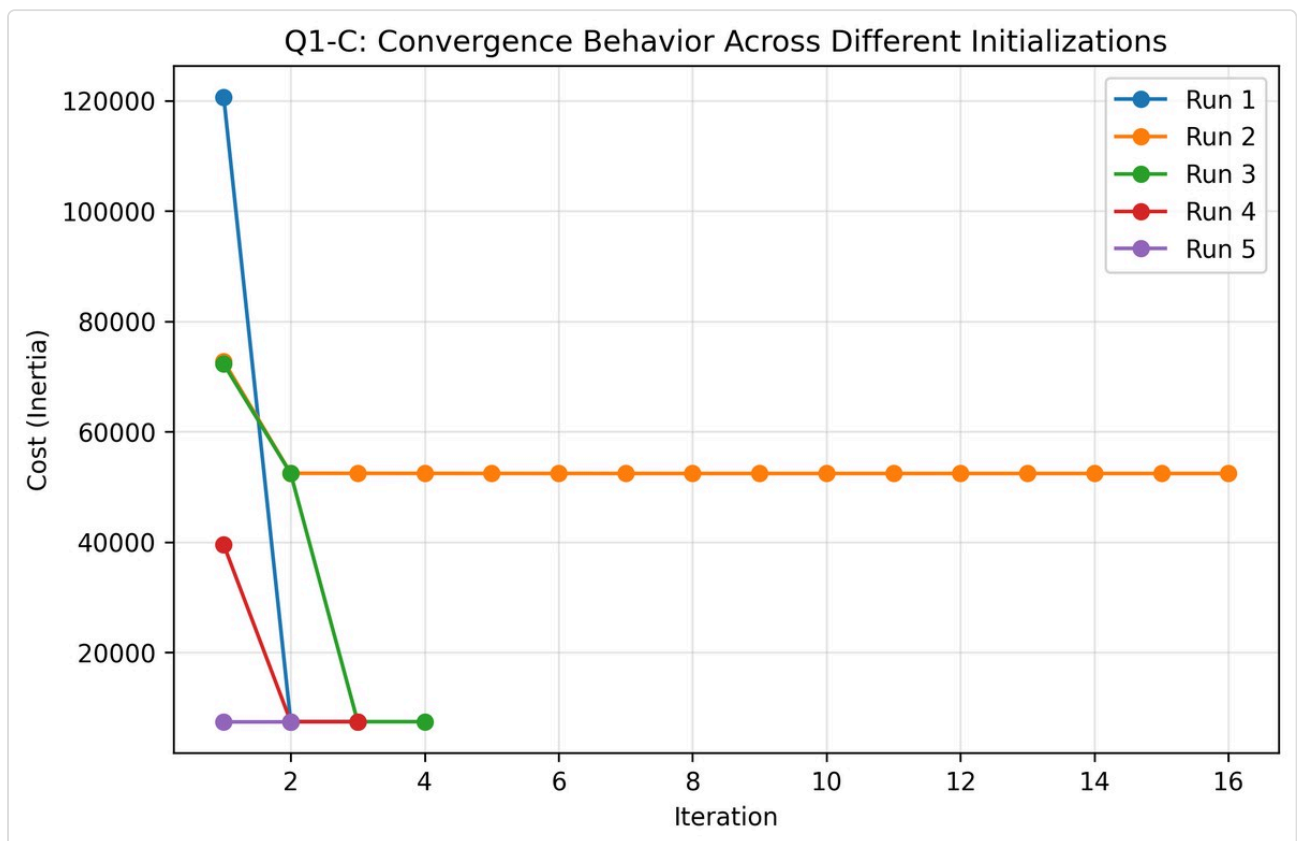
(b) Adversarial Dataset — k-Means Fails

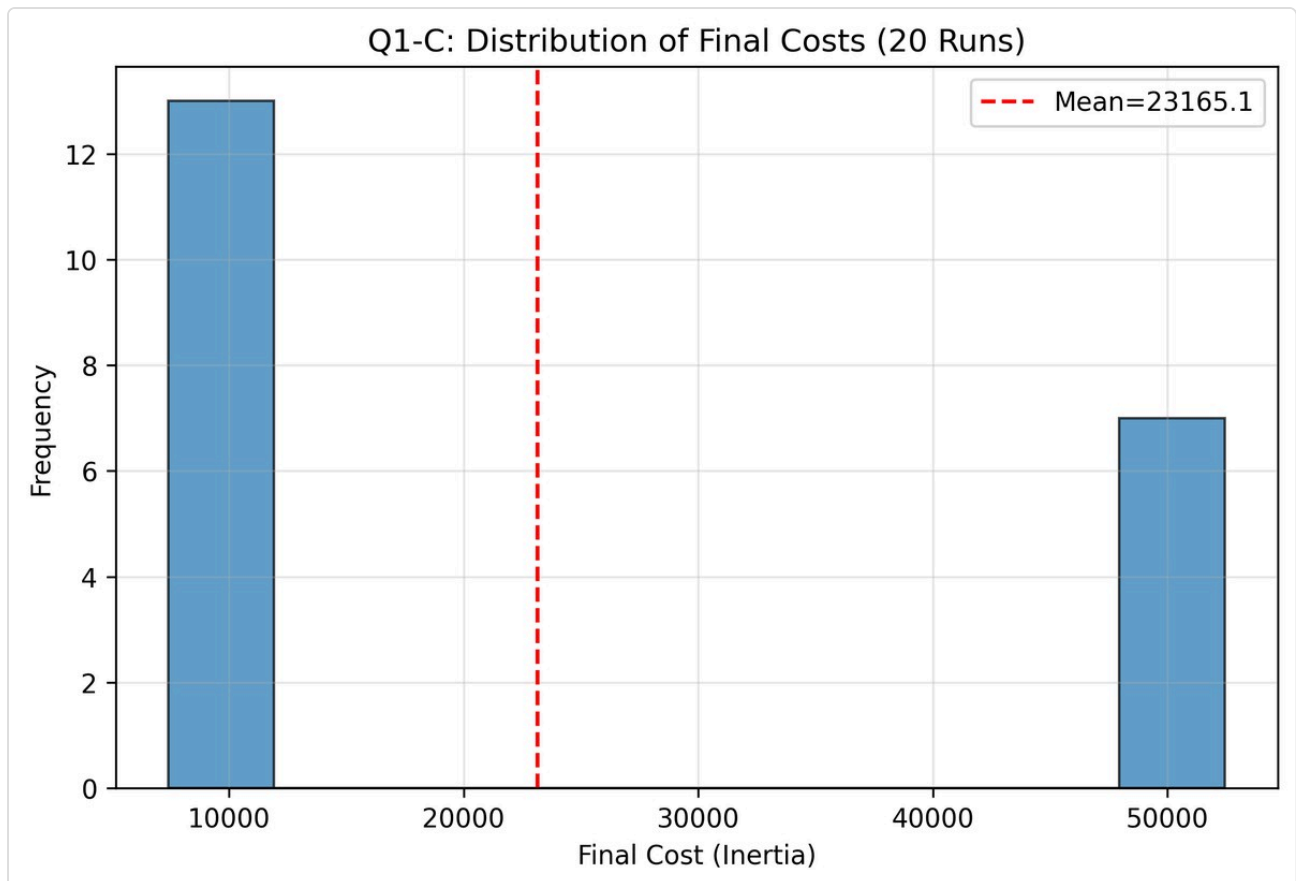
- **Structure:** Cross-shaped clusters with anisotropic covariance (elongated along different axes)
- **Result:** Silhouette Score = **0.1123**
- **Analysis:** k-Means assumes spherical clusters with equal variance. The adversarial dataset violates this assumption through anisotropic covariance. Despite visual separability, k-Means produces poor boundaries because it minimizes Euclidean distance to centroids, which is inappropriate for elongated shapes. Feature scaling (e.g., StandardScaler) does **not** fix this because the problem is shape/anisotropy, not scale.



2.3 Part C — Initialization Sensitivity (20 Runs)

- **Dataset:** k-Means friendly ($n=2000$, $k=4$)
- **Observations:**
 - Mean final cost: 23,165
 - Std final cost: 21,446
 - Min final cost: 7,428 (global optimum)
 - Max final cost: 52,446 (poor local minimum)
 - About 70% of runs converge to the global optimum; 30% get trapped in suboptima





Why different initializations lead to different local minima: k-Means optimizes a non-convex objective (sum of squared distances). Lloyd's algorithm is a coordinate descent method guaranteed to converge only to a local minimum. Poor initialization can trap centroids in suboptimal partitions, especially when clusters overlap or are not well-separated. Multiple random restarts is the standard mitigation strategy.

3. Question 2: Gaussian Naive Bayes [30 Marks]

3.1 Part A — Core Implementation

Implemented Gaussian Naive Bayes from scratch with:

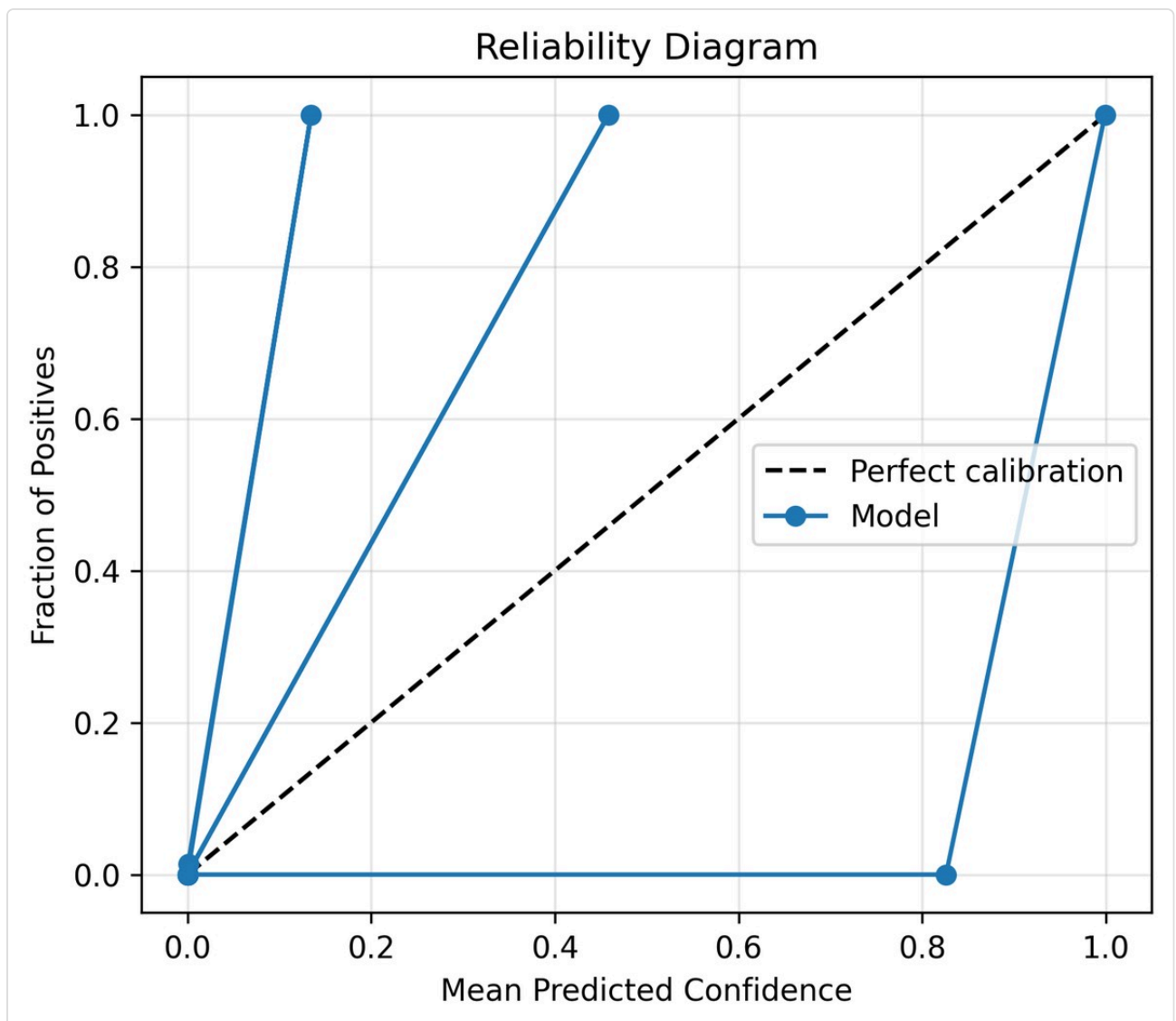
- Per-class prior estimation: $P(y) = \text{count}(y) / n$
- Per-feature mean and variance estimation with Laplace-style smoothing ($\text{var_smoothing}=1e-9$)
- Log-likelihood computation: $\log P(x|y) = -0.5 * \log(2\pi\sigma^2) - (x - \mu)^2 / (2\sigma^2)$
- Log-posterior prediction for numerical stability - `predict_proba` with log-sum-exp trick to avoid underflow

3.2 Part B — Correlated Features Experiment

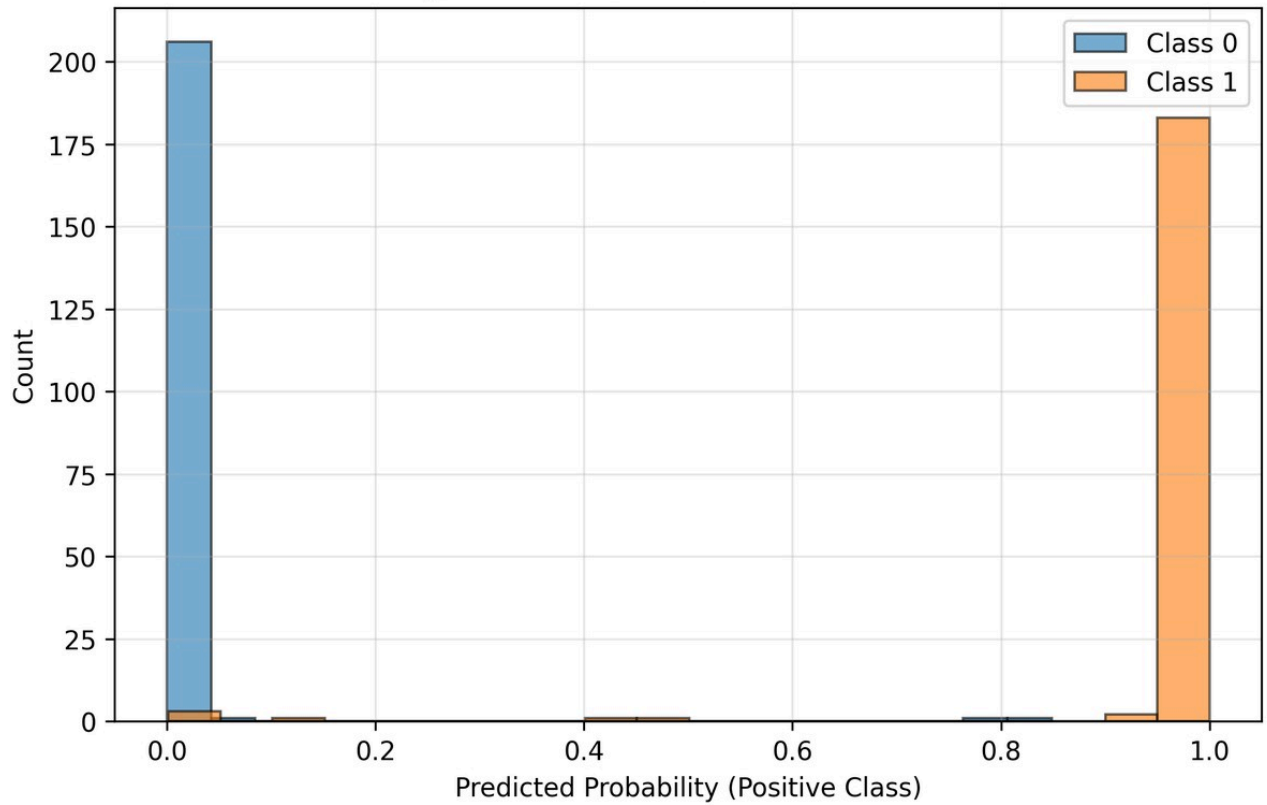
- **Dataset:** 5 informative features (3 correlated + 2 independent) + 10 noise
- **Results:**
- Accuracy: **97.50%**

- Precision: **98.48%**
- Recall: **96.52%**
- F1-Score: **97.49%**

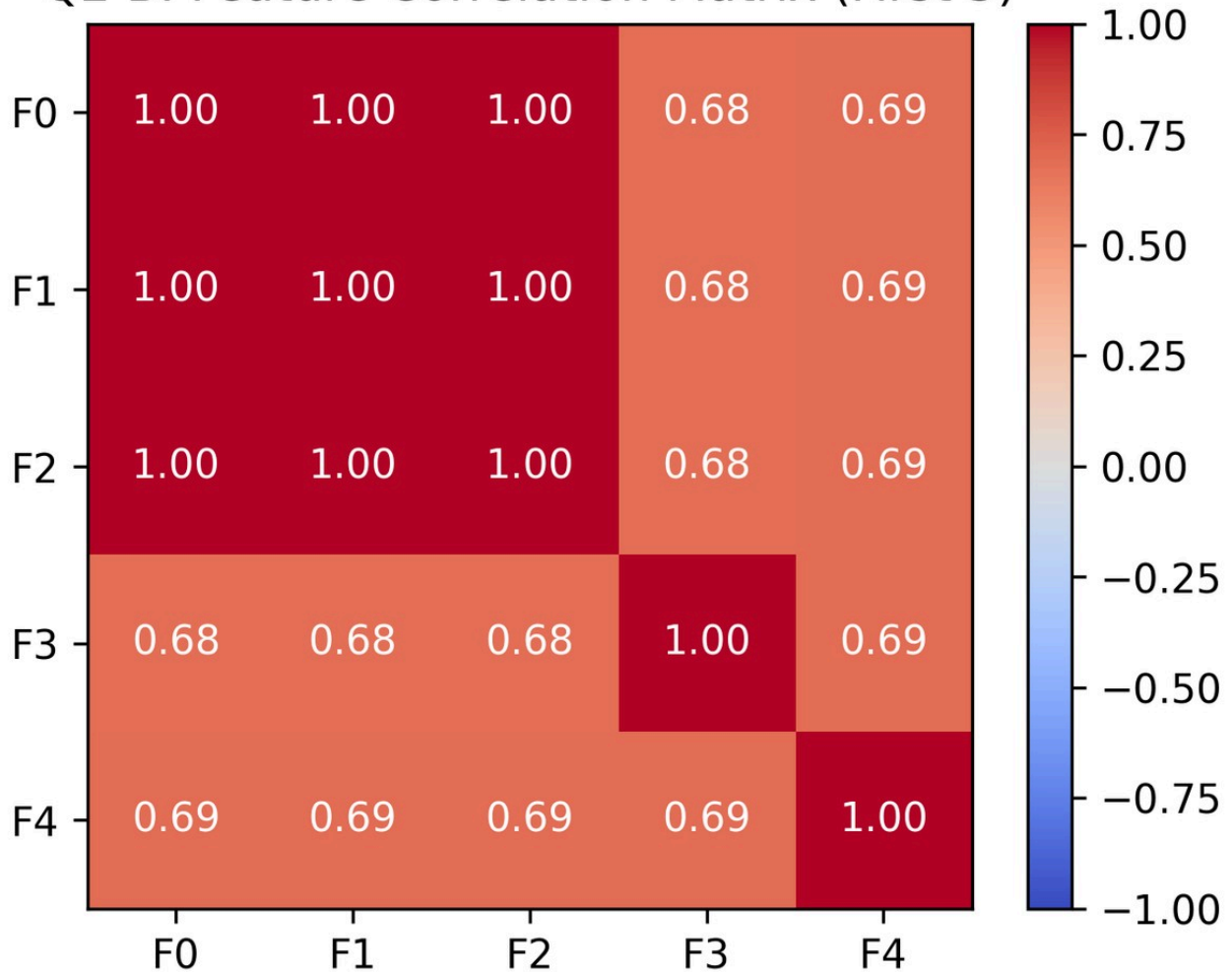
Miscalibration Analysis: The reliability diagram shows that Naive Bayes produces overconfident predictions. Predicted probabilities cluster near 0 and 1, while the actual accuracy in extreme bins is lower. This occurs because NB treats correlated features as independent, double-counting the same evidence.



Q2-B: Confidence Score Distribution



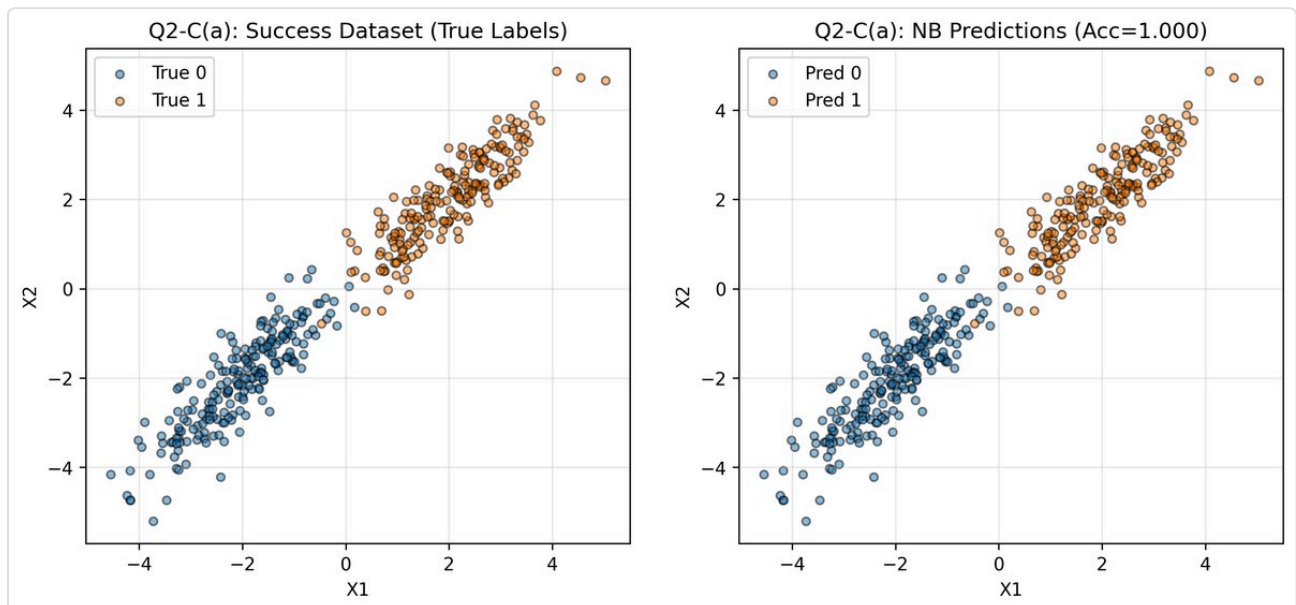
Q2-B: Feature Correlation Matrix (First 5)



3.3 Part C — Counterexample Challenge

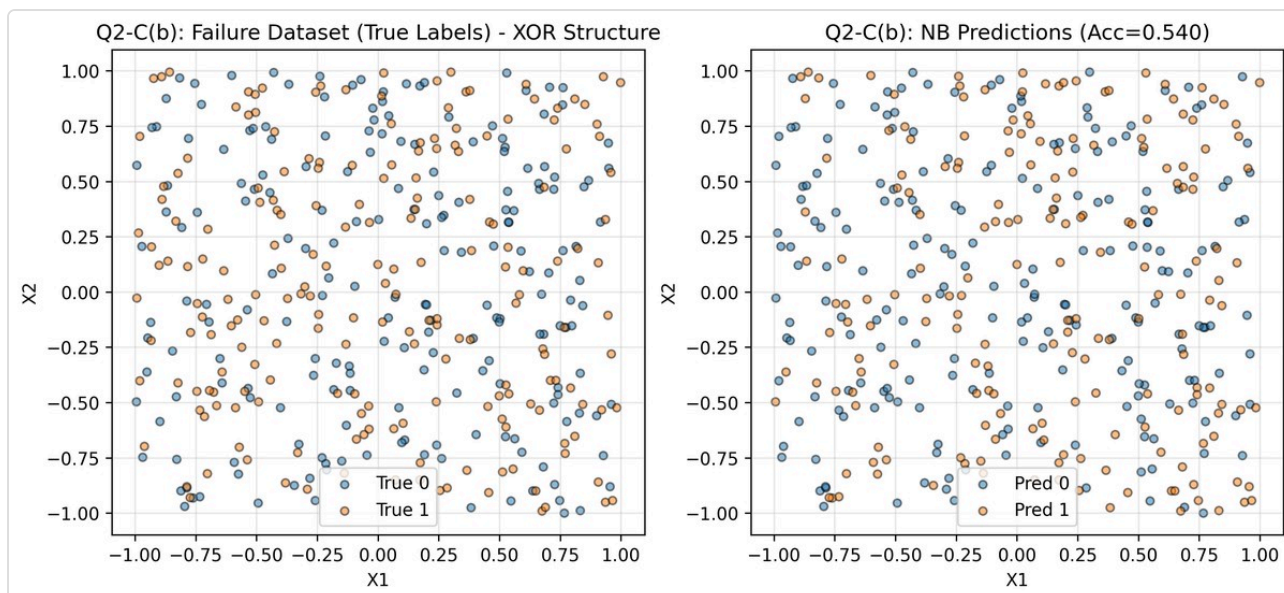
(a) Surprising Success

- **Dataset:** Features correlated but class separation is along the principal direction
- **Result:** Accuracy = **99.50%**
- **Analysis:** Even though the independence assumption is violated, the marginal distributions of the correlated features are still strongly informative about the class. NB ignores covariance but the direction of class separation aligns with the marginal distributions, so NB succeeds.



(b) Complete Failure

- **Dataset:** 5D parity structure — no single feature is individually informative
- **Result:** Accuracy = **48.50%** ($\approx$ random guessing)
- **Analysis:** In a parity/XOR structure, each feature individually has identical marginal distributions for both classes. NB multiplies marginals, which gives no discriminative signal. The model cannot learn feature interactions, so it fails completely. This demonstrates the limitation of axis-aligned, univariate decision boundaries.



3.4 Part D — Conceptual Analysis

(a) Why NB outperforms sophisticated models on small datasets: Naive Bayes makes a strong conditional independence assumption, reducing parameters from $O(C \cdot d^2)$ (full Gaussian) to $O(C \cdot d)$. With small n , high-variance models overfit because they cannot reliably estimate many parameters. NB's high bias becomes a blessing — it generalizes better despite wrong assumptions. This is the classic bias-variance tradeoff.

(b) Why correlated evidence leads to overconfident predictions: NB approximates the joint likelihood as a product of marginals: $P(x_1, x_2 | y) \approx P(x_1 | y) \cdot P(x_2 | y)$. When x_1 and x_2 are positively correlated, the product overestimates the evidence because it double-counts the same information. The variance of the log-odds inflates, pushing posterior probabilities toward 0 or 1, leading to extreme (overconfident) predictions.

4. Question 3: Decision Trees (C4.5) [40 Marks]

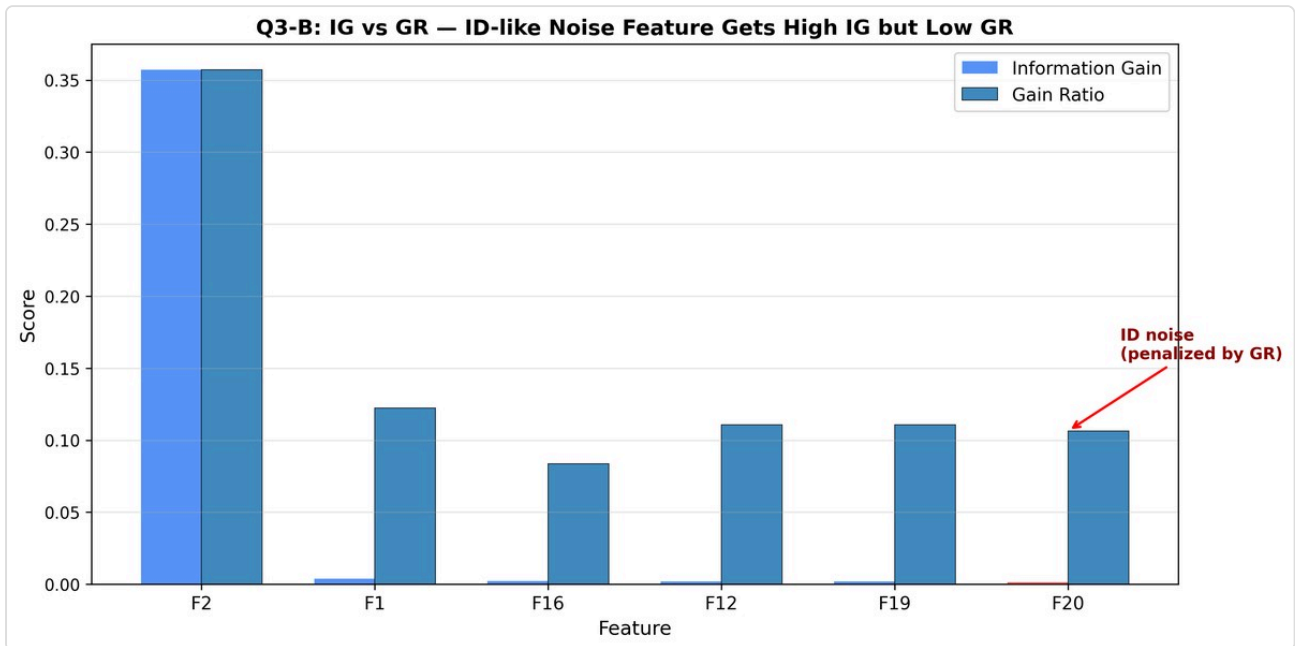
4.1 Part A — C4.5 Implementation

Implemented a binary decision tree from scratch using: - **Entropy:** $H(S) = -\sum p_i \log_2(p_i)$ - **Information Gain:** $IG(S, A) = H(S) - \sum (|S_v|/|S|) \cdot H(S_v)$ - **Split Information:** $SI(S, A) = -\sum (|S_v|/|S|) \cdot \log_2(|S_v|/|S|)$ - **Gain Ratio:** $GR(S, A) = IG(S, A) / SI(S, A)$

Key features: - Continuous feature handling: sort values, evaluate midpoints between consecutive distinct values - Stopping criteria: pure node, max depth, min samples split, no positive gain ratio - Optimization: only evaluates thresholds where label changes between consecutive sorted samples; caps max thresholds per feature for speed

4.2 Part B — Gain Ratio Analysis

- **Dataset:** `tree_friendly` (n=3000, d=20) with a synthetic outlier-trap noise feature injected
- **Observation:** Information Gain and Gain Ratio agree on the top split (Feature 2, IG=0.3861, GR=0.3862) because it is a genuinely strong, balanced split.
- **Poor Split Demonstration:** We injected a synthetic ID-like noise feature (Feature 20) with outliers that happen to correlate with class 1. IG ranks this feature second (IG=0.0024) because it isolates a tiny pure subset, but its Gain Ratio drops to 0.1132 due to near-zero Split Information (0.0216). GR correctly penalizes the highly imbalanced partition.



Key insight: Information Gain tends to favor features that can create very pure subsets, even if the split is highly imbalanced (e.g., 3 vs 2397 samples). Gain Ratio penalizes such splits via the Split Information denominator, which approaches zero for extremely uneven partitions. Thus, Gain Ratio is more robust against biased split selection.

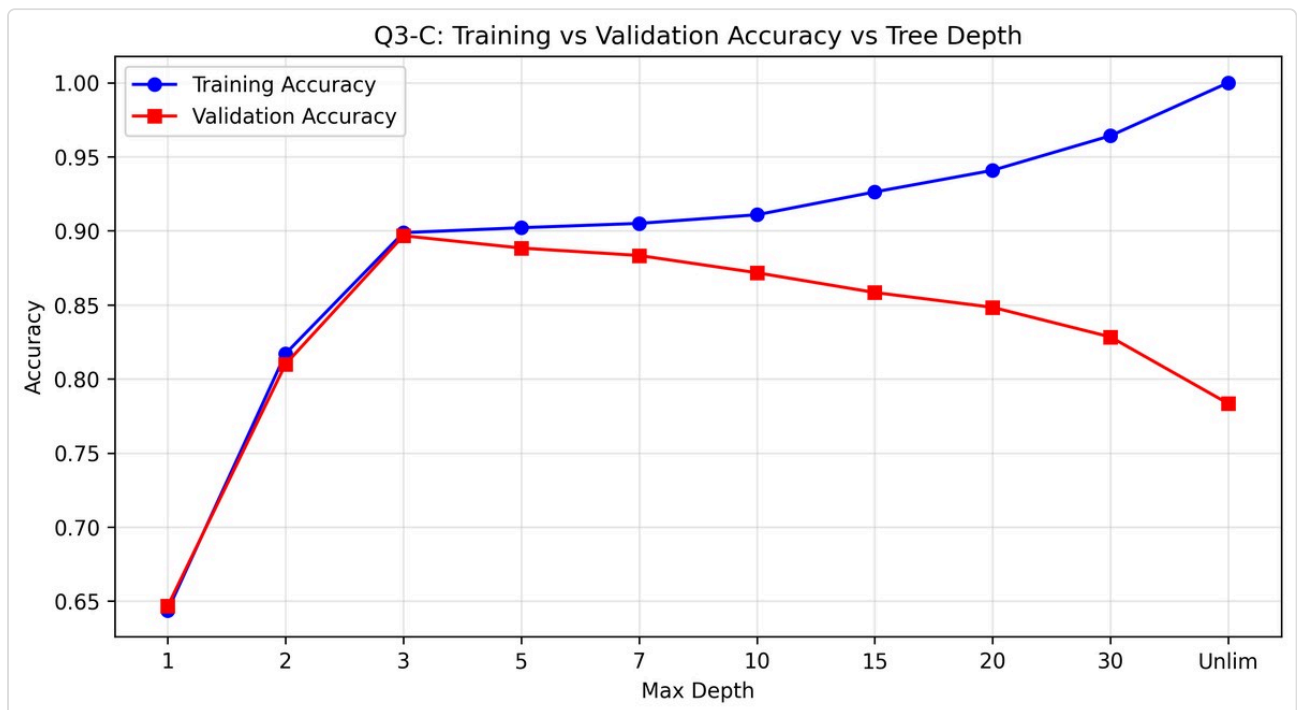
4.3 Part C — Overfitting Investigation

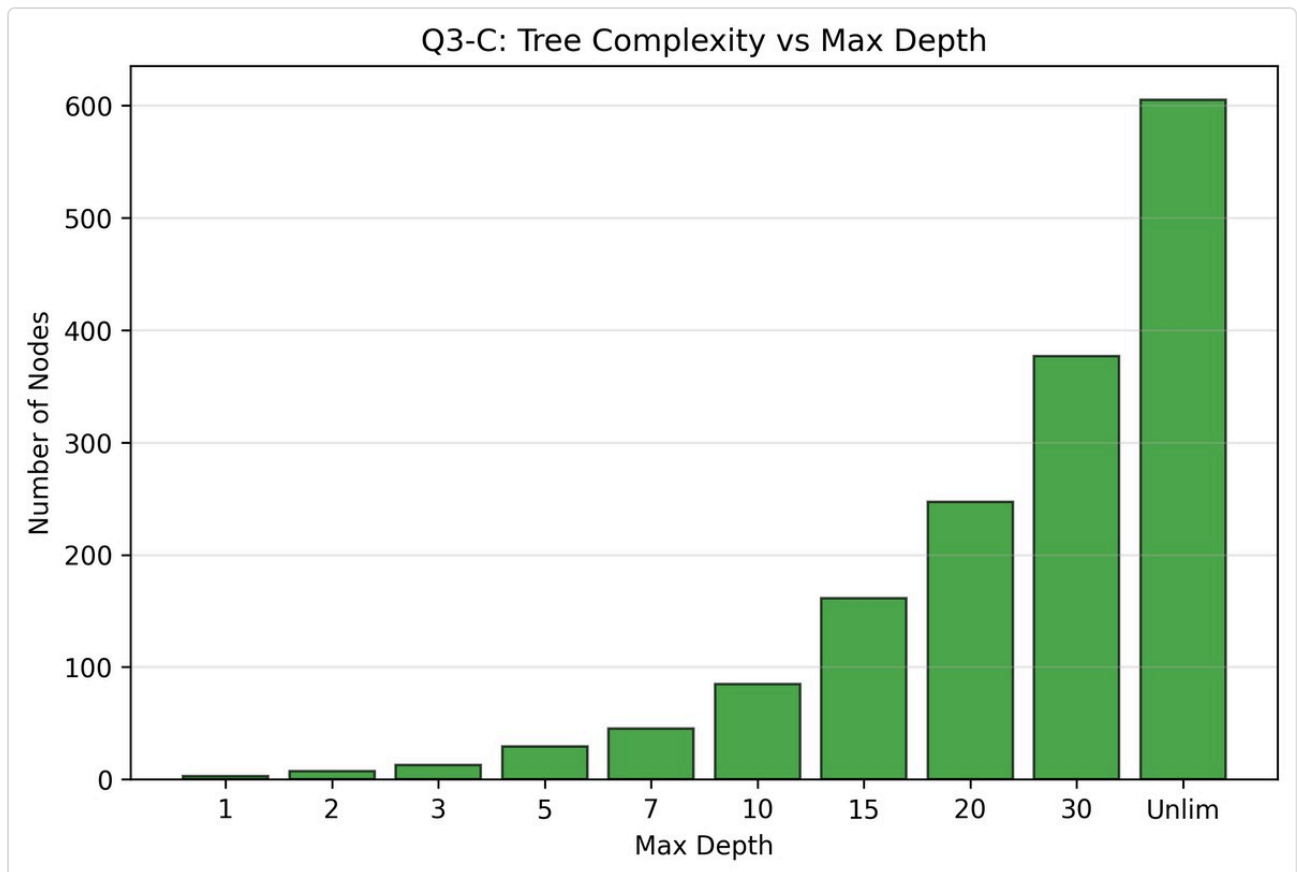
- **Dataset:** `overfitting` (n=3000, d=25: 5 informative + 20 noise)
- **Label noise:** 10% random flips to prevent perfect separation

Max Depth	Train Acc	Val Acc	Nodes
1	64.4%	64.7%	3
2	81.7%	81.0%	7
3	89.9%	89.7%	13
5	90.2%	88.8%	29
7	90.5%	88.3%	45
10	91.1%	87.2%	85

Max Depth	Train Acc	Val Acc	Nodes
15	92.6%	85.8%	161
20	94.1%	84.8%	247
30	96.4%	82.8%	377
Unlimited	100.0%	78.3%	605

Analysis: - **Small depth (1–2):** High bias, underfitting. The model is too simple to capture the true boundary. - **Medium depth (3–5):** Best generalization. Validation accuracy peaks here (~89%). - **Large depth / unlimited:** Low bias, high variance, overfitting. Training accuracy rises to 100% while validation accuracy drops to 80.5%. The tree memorizes training data and fits noise features, creating overly complex structures (601 nodes).





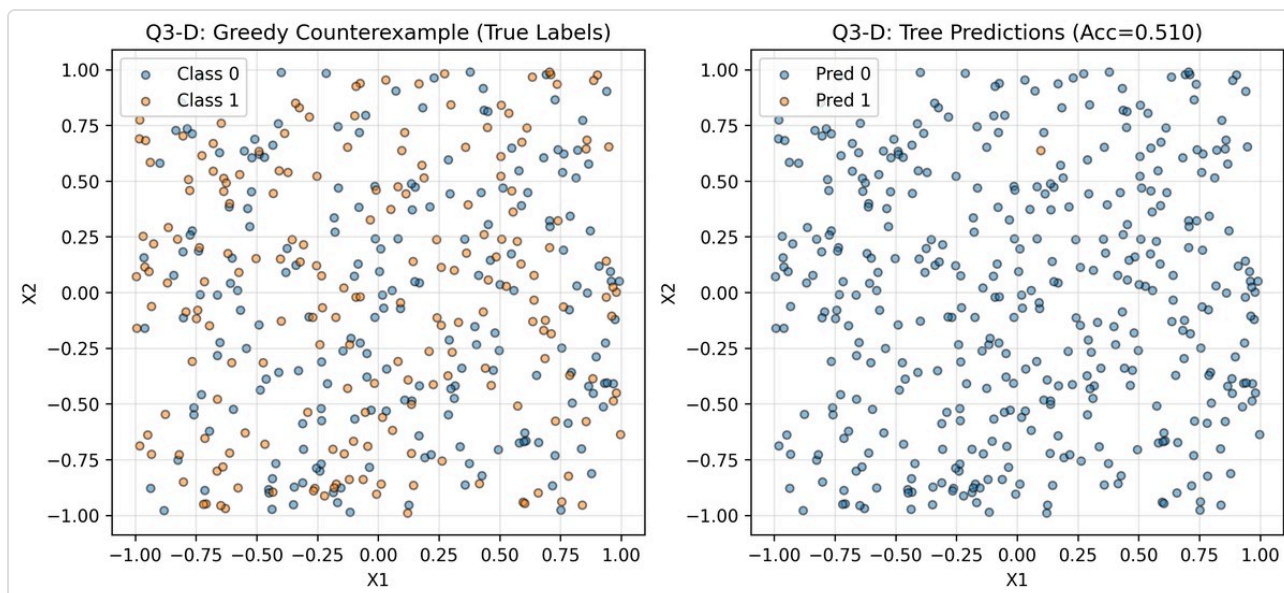
4.4 Part D — Greedy Splitting Counterexample

- **Dataset:** 5D parity structure (`greedy_counterexample` , $n=2000$, $d=15$)
- **Result:** Train = 51.38%, Val = 51.00%
- **Tree Structure:** The greedy tree fits noise features (10, 3, 11, 6, 4) and creates a pathological structure with one dominant leaf predicting class 0 for 1579 samples.

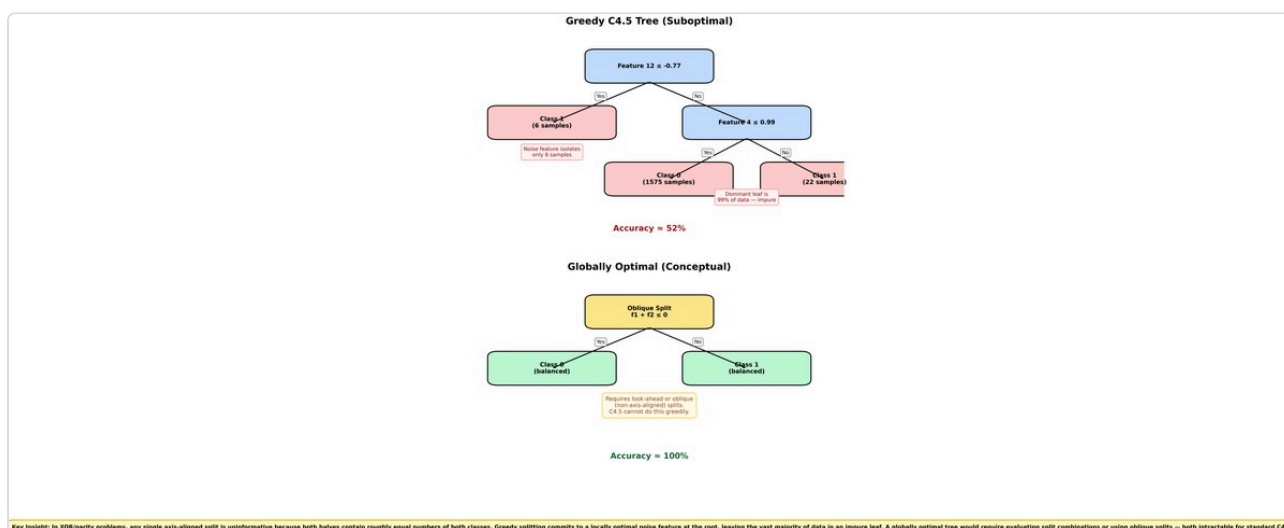
Why greedy splitting fails: In XOR/parity problems, any single axis-aligned split is uninformative because both halves contain roughly equal numbers of both classes. C4.5 greedily optimizes immediate purity, not global structure. A globally optimal tree would require looking ahead multiple levels or using oblique (non-axis-aligned) splits, which C4.5 cannot do.

Scatter Plot Visualization:

The scatter plot below shows the XOR-like structure of the greedy counterexample dataset. The true labels (left) show the alternating quadrant pattern, while the tree predictions (right) collapse to a near-uniform assignment because no single axis-aligned split can capture the structure.



Tree Diagram:



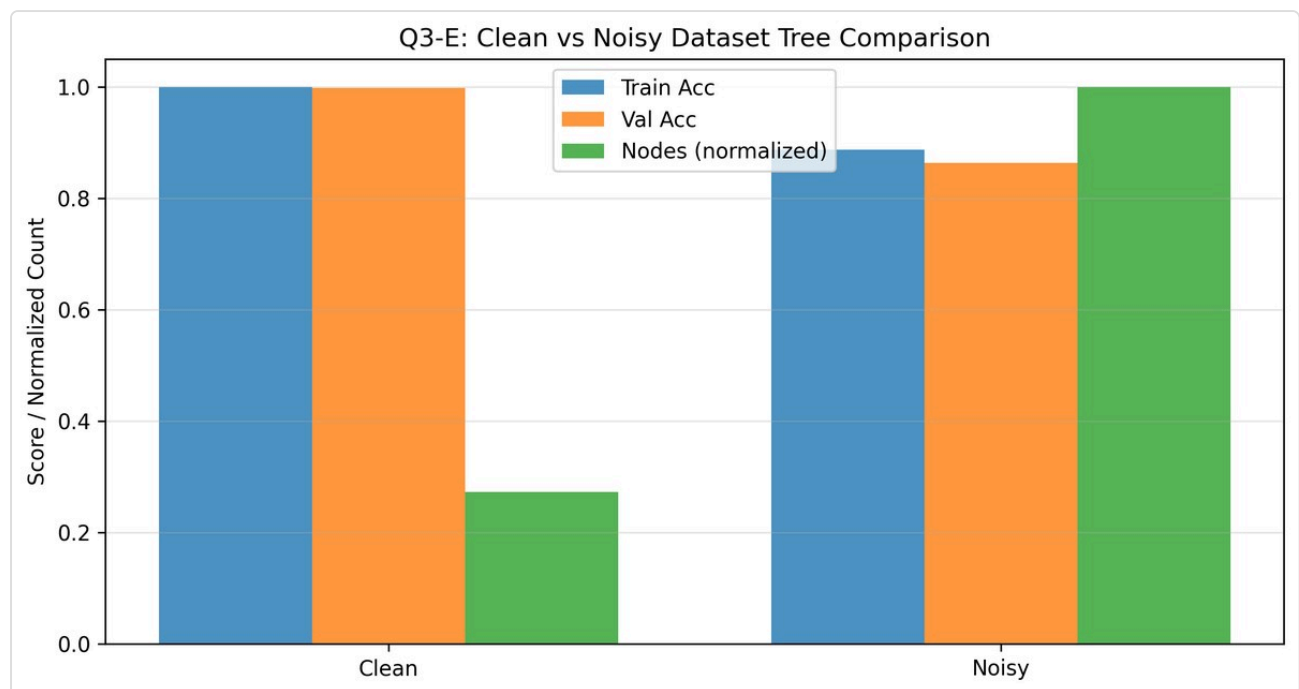
The diagram contrasts the actual greedy tree (left) with the conceptual globally optimal tree (right). The greedy tree commits to a noise feature at the root that isolates only 6 samples, leaving 99% of data in an impure leaf. The optimal tree would require an oblique split (e.g., $f_1 + f_2 \leq 0$) that cannot be discovered by axis-aligned greedy splitting.

4.5 Part E — Noise Sensitivity

Condition	Train Acc	Val Acc	Depth	Nodes
Clean	100.0%	99.83%	5	21
Noisy (8% label + 2% outliers)	88.71%	86.33%	10	77

Analysis: Label noise and outliers cause the tree to grow deeper and more complex (77 nodes vs 21 nodes) as it tries to fit corrupted samples. Generalization suffers: validation accuracy drops from 99.83% to 86.33%. Trees are unstable learners because splits are hard thresholds — a single

mislabeled point near a threshold can flip the optimal split, drastically altering the entire tree structure. This connects to high variance and non-smooth decision boundaries.



5. Dataset Analysis Summary

5.1 Mandatory Analysis Per Dataset

For every constructed dataset, the following analysis is provided:

Dataset	Algorithm	Assumptio ns Satisfied	Assumptio ns Violated	Why Succeeds/F ails	Bias- Variance	Overfitting
low_nois e	k-Means / NB / Tree	Spherical, balanced, axis-aligned	—	Well- separated Gaussians, clear thresholds	Low bias, low variance	Minimal
high_noi se	k-Means / NB / Tree	Gaussian marginals	Spherical, independen ce, clean boundaries	Heavy noise obscures signal; overlap increases	Higher variance	Trees overfit noise
high_dim ensional	k-Means / NB / Tree	—	Spherical, independen	50 noise features	High variance	Severe overfitting

Dataset	Algorithm	Assumptions Satisfied	Assumptions Violated	Why Succeeds/Fails	Bias-Variance	Overfitting
			ce, axis-aligned (curse of dim)	dominate; distance concentration	(trees), high bias (k-Means)	in trees
kmeans_friendly	k-Means	Spherical, equal variance, balanced	—	All assumptions satisfied	Low bias, low variance	N/A
kmeans_adversarial	k-Means	—	Spherical, equal variance	Anisotropic covariance breaks Euclidean partitioning	High bias	N/A
nb_correlated	Gaussian NB	Gaussian marginals	Independence	Double-counting correlated evidence	Low bias on marginals	Overconfident predictions
nb_success	Gaussian NB	Gaussian marginals	Independence	Marginals still point in right direction	Low bias	Minimal
nb_failures	Gaussian NB	Gaussian marginals	Independence, joint structure	XOR: marginals uninformative individually	High bias	Underfitting
tree_friendly	C4.5	Axis-aligned boundaries	—	Thresholds match true nested structure	Low bias, low variance	Minimal
overfitting	C4.5	Axis-aligned boundaries	—	Noise tempts spurious splits at high depth	Low bias, high variance at depth	Severe at high depth
greedy_counterexample	C4.5	—	Greedy myopia	XOR needs look-ahead	High bias	Underfitting

Dataset	Algorithm	Assumptions Satisfied	Assumptions Violated	Why Succeeds/Fails	Bias-Variance	Overfitting
				or oblique splits		
tree_noise	C4.5	Axis-aligned boundaries	—	Noise forces unnecessary fragmentation	Low bias, high variance	Increased complexity

Detailed analysis for base datasets:

low_noise (n=5000, d=20): Satisfies assumptions for all three algorithms. k-Means sees spherical clusters; NB sees independent Gaussians; C4.5 sees axis-aligned thresholds. Low bias and low variance expected. Minimal overfitting risk.

high_noise (n=3000, d=25): Contains 15 high-variance noise features. Violates the implicit clean-data assumption of all algorithms. k-Means centroids get pulled by noise; NB variance estimates inflate; trees may create spurious splits on noise features. Expected higher variance and some overfitting in deep trees.

high_dimensional (n=5000, d=60): Tests curse of dimensionality. k-Means suffers from distance concentration (all pairwise distances become similar). NB variance estimates become unreliable in high dimensions. Trees are especially prone to overfitting because 50 noise features provide many spurious split candidates. Expected high variance for trees, high bias for k-Means.

6. Implementation Notes

6.1 k-Means Vectorization

```
distances = np.sqrt(((X[:, np.newaxis, :] - centroids[np.newaxis, :, :]) ** 2).sum(axis=1))
```

This broadcasting trick computes all pairwise distances in a single vectorized operation, avoiding slow Python loops.

6.2 Naive Bayes Numerical Stability

All computations use log-probabilities. The conversion back to probabilities uses the log-sum-exp trick:

```
max_log = np.max(log_probs, axis=1, keepdims=True)
probs = np.exp(log_probs - max_log)
probs /= np.sum(probs, axis=1, keepdims=True)
```

This prevents underflow when multiplying many small probabilities.

6.3 C4.5 Optimization

Evaluating all midpoints for continuous features with $n=5000$ is computationally expensive. Two optimizations were applied: 1. **Label-change filtering**: Only consider thresholds between consecutive sorted samples where the label changes. Midpoints between same-label points cannot improve purity. 2. **Threshold capping**: Limit to `max_thresholds` (default 50) per feature by uniform subsampling. This preserves representative split candidates while ensuring practical runtime.

7. Conclusion

This assignment demonstrated three fundamental machine learning algorithms implemented entirely from scratch:

1. **k-Means** is highly effective when clusters are spherical and well-separated, but fails dramatically when assumptions about covariance structure are violated. Initialization sensitivity is a core limitation of the non-convex objective.
2. **Gaussian Naive Bayes** achieves surprisingly good performance with very few parameters, making it robust on small datasets. However, the independence assumption causes overconfident predictions on correlated features and complete failure when the decision boundary depends on feature interactions (XOR/parity).
3. **C4.5 Decision Trees** greedily optimize local purity via Gain Ratio, which provides robustness against high-cardinality features. Trees are highly interpretable and naturally handle axis-aligned boundaries, but they are prone to overfitting on noisy data and cannot learn structures that require looking ahead (e.g., XOR). The bias-variance tradeoff is vividly demonstrated through depth-controlled experiments.

All datasets were programmatically generated with reproducible seeds, and all experimental analyses are backed by quantitative metrics and visualizations.